语义分割课程实验报告

CamVid 11 类语义分割：五种网络结构的训练与测试对比

1. 任务

在课程给出的残差 U-Net 上做 CamVid 多类语义分割。每个像素分成 11 类，相当于对像素做多类分类。本次在同一数据划分、同一损失和同一训练日程下比较五种结构，用验证集 mIoU 保存权重，再在未参与选模的测试集上比较分类能力。

代码名	结构	和其余模型的差别
`unet`	残差 U-Net	四级跳跃连接，解码器拼接编码器特征
`fcn`	无跳跃连接的编码器-解码器	同一编码器，解码器只做转置卷积上采样
`segnet`	SegNet	用最大池化的位置索引做反池化
`deeplab`	DeepLabV3+	瓶颈处做空洞空间金字塔，再接一条低层特征
`attunet`	Attention U-Net	跳跃连接拼接前先乘空间注意力

五种网络共用残差卷积块，通道都是 64、128、256、512、1024。差别集中在特征如何回到原分辨率，以及要不要多尺度上下文。

2. 数据集

使用 CamVid 驾驶场景数据集。图像来自车载摄像头，共 701 张 960×720 彩色图及标注。原始 32 类按 SegNet 的常用规则合并为 11 类。Void 的编号是 255，不进入损失，也不进入指标。

合并后类别	包含的原始类
天空	Sky
建筑	Building, Wall, Archway, Bridge, Tunnel
杆柱	Column_Pole, TrafficCone
道路	Road, LaneMkgsDriv, LaneMkgsNonDriv
人行道	Sidewalk, ParkingBlock, RoadShoulder
树木	Tree, VegetationMisc
标志	SignSymbol, Misc_Text, TrafficLight
围栏	Fence
车辆	Car, SUVPickupTruck, Truck_Bus, Train, OtherMoving
行人	Pedestrian, Child, CartLuggagePram, Animal
骑行者	Bicyclist, MotorcycleScooter

划分：`valid.txt` 中的 101 张全部来自 0016E5 这一段行驶序列，作为测试集。其余 600 张按随机种子 42 打乱，90 张做验证集，用来保存验证 mIoU 最高的权重，510 张用于训练。

图像从 960×720 双线性缩放到 320×240，标注用最近邻缩放。训练集使用水平翻转、±12° 旋转和亮度、对比度抖动。旋转在标注上留出的空白填 255。像素缩放到 [0, 1]，编码器没有使用 ImageNet 预训练。

类别权重是 median frequency balancing，w_c = median(f) / f_c。道路最多，权重 0.192；骑行者最少，权重 20.306。

3. 网络结构

输入都是 3×240×320。四次 2×2 最大池化后，空间尺寸依次为 120×160、60×80、30×40，瓶颈为 15×20。输出是 11 个类别的 logits，预测时对通道做 argmax。实现位于 `model/net.py`。

3.1 残差 U-Net

编码器通道为 64、128、256、512，瓶颈为 1024。解码器用步长为 2 的转置卷积把分辨率放大一倍，再与编码器同尺度特征在通道维拼接，然后进入残差块。因为拼接，解码器每一级的输入通道是该级通道数的两倍。最后用 1×1 卷积得到 11 类 logits。可训练参数为 32,428,171。

放大后的特征提供语义，同尺度的编码器特征保留池化前的边缘和位置。

3.2 无跳跃连接的编码器-解码器

编码器与残差 U-Net 相同。解码器不拼接编码器特征，只处理上一层转置卷积的结果，因此每一级解码器的输入通道不加倍。它逐级放大，两侧的差别集中在有没有跳跃连接。代码入口名称是 `fcn`。可训练参数为 28,597,451。

3.3 SegNet

SegNet 记住每次最大池化选中的位置，上采样时用 MaxUnpool2d 把激活放回这些位置，其余位置填 0，再做卷积。它不拼接编码器的特征值。瓶颈先把通道扩到 1024，反池化要求通道数和当初被池化的特征一致，所以先用 1×1 卷积压回 512，再按索引放大。可训练参数为 22,636,875，五种里最少。

索引反池化把响应当时最强的位置还回去，适合杆柱这类细目标。被池化丢掉的其他位置没有编码器特征可补，边界主要靠后面的卷积重新长出来。

3.4 DeepLabV3+

编码器仍是同一套残差块。瓶颈接空洞空间金字塔（ASPP）：一条 1×1 卷积、三条膨胀率分别为 2、4、6 的 3×3 卷积，以及一张全局平均池化。五路特征拼在一起，再用 1×1 卷积压到 256 通道，并带 BatchNorm 和 Dropout（比率 0.1）。

论文里的膨胀率 6、12、18 是为更大的特征图准备的。这里瓶颈只有 15×20，膨胀率 12 的感受野已经超出特征图，所以改成 2、4、6。图像级池化仍然提供整图上下文。

解码器把 ASPP 输出双线性放大到 60×80，与第三次卷积后的低层特征拼接。低层特征先用 1×1 卷积压到 48 通道。融合后再放大到 240×320。低层特征只接这一次，位置在原图的 1/4 分辨率。可训练参数为 29,332,779。

ASPP 和解码器里的 BatchNorm 是这一种网络独有的，其余四种没有归一化层。比较时把这一点算进 DeepLabV3+ 的结构设计。

3.5 Attention U-Net

骨架与残差 U-Net 相同。每一级跳跃连接在拼接之前经过注意力门：把编码器特征和解码器上采样特征各自做 1×1 卷积，相加后 ReLU，再映射成一张单通道图，sigmoid 之后乘在编码器特征上。权重接近 1 的位置被保留，接近 0 的位置被压低。可训练参数为 32,776,815，五种里最多。

4. 训练设置

五种模型使用同一套超参数，分别执行 `python main.py --model <名称>`。选模只用验证集。测试在训练结束后用 best.pth 计算。

项目	取值
输入	3×240×320，像素值在 [0, 1]
输出	11 类 logits
优化器	AdamW，学习率 3×10^{-4}，权重衰减 1×10^{-4}
学习率调度	余弦退火，周期等于总轮数
损失	加权交叉熵，忽略编号 255
梯度裁剪	全局范数上限 1.0
数值精度	CUDA 上使用 bfloat16
batch size / 轮数 / 种子	8 / 40 / 42
设备	NVIDIA RTX 5090

$$L = -\Sigma_{c=0..10}\, w_c\, y_c \log p_c$$

y_c 是该像素的 one-hot 标签，p_c 是 softmax 后的概率，w_c 是类别权重。

验证集上 mIoU 最高的一轮：

模型	轮次	验证损失	验证 mIoU	验证 PA
残差 U-Net	33	0.8316	0.4500	0.7857
无跳跃连接	33	0.8688	0.4425	0.7810
SegNet	33	0.9086	0.4186	0.7473
DeepLabV3+	33	0.5711	**0.5238**	**0.8449**
Attention U-Net	37	0.7978	0.4449	0.7672

权重保存在 `runs/<模型名>/best.pth`。DeepLabV3+ 在第 33 轮的验证 mIoU 为 0.5238。其余四个模型的验证 mIoU 集中在 0.42 到 0.45。

5. 指标

测试集上把全部有效像素累加成混淆矩阵 M。行是真实类别，列是预测类别。Void 不进入矩阵。TP_i 是对角线，GT_i 是第 i 行的和，$Pred_i$ 是第 i 列的和，N 是全部有效像素。

指标	计算	作用
$Precision_i$	TP_i / $Pred_i$	预测为该类的像素里有多少是对的
$Recall_i$	TP_i / GT_i	该类真实像素里有多少被找回来
IoU_i	TP_i / (GT_i + $Pred_i$ − TP_i)	预测区域与真实区域的交并比
$Dice_i$	2 TP_i / (GT_i + $Pred_i$)	与 IoU 单调对应
PA	Σ TP_i / N	全部有效像素的准确率
mPA	各类召回率的平均	每个类一票
mIoU	各类 IoU 的平均	模型对比以它为准
mDice	各类 Dice 的平均	与 mIoU 一起看重叠
FWIoU	Σ (GT_i / N) · IoU_i	按真实像素占比加权的 IoU

本次测试集 11 类都有像素。PA 和 FWIoU 受道路、天空、建筑这类大区域支配。mIoU 对杆柱、行人、标志同样敏感，所以用 mIoU 给模型排名，再用精确率和召回率看某个类是漏检还是误检。

6. 测试集总指标

以下数字来自五个 best.pth 在 101 张测试图上的同一次评估，汇总文件是 `runs/figures/summary.json`。各模型自己的结果同时写在 `runs/<模型名>/test.metrics.json`。

模型	参数量	PA	mPA	mIoU	FWIoU	mDice
残差 U-Net	32.43M	0.7563	0.7089	0.4710	0.6686	0.5843
无跳跃连接	28.60M	0.7376	0.6884	0.4608	0.6598	0.5623
SegNet	22.64M	0.7238	0.6895	0.4491	0.6422	0.5584
DeepLabV3+	29.33M	**0.8227**	**0.7571**	**0.5239**	**0.7548**	**0.6339**
Attention U-Net	32.78M	0.7285	0.6863	0.4555	0.6419	0.5659

按 mIoU 从高到低：DeepLabV3+ 0.5239，残差 U-Net 0.4710，无跳跃连接 0.4608，Attention U-Net 0.4555，SegNet 0.4491。

DeepLabV3+ 比残差 U-Net 高 0.0529 个 mIoU，像素准确率高 0.0664。它的参数量还少约 310 万。

验证 mIoU 与测试 mIoU 的排序一致：DeepLabV3+ 验证 0.5238、测试 0.5239；残差 U-Net 验证 0.4500、测试 0.4710。测试集是同一段视频的连续帧，验证集来自其他序列，两边接近，说明优势在另一段路上同样存在。

7. 各类 IoU

类别	残差 U-Net	无跳跃连接	SegNet	DeepLabV3+	Attention U-Net
天空	0.9381	0.9052	0.9179	0.9286	**0.9415**
建筑	0.4433	0.3432	0.3342	**0.5971**	0.3644
杆柱	0.0235	0.0093	0.0310	**0.0500**	0.0223
道路	0.8672	0.8649	0.8786	**0.9181**	0.8626
人行道	0.6744	0.7023	0.6965	**0.7699**	0.6730
树木	0.7244	0.8426	0.7361	**0.8552**	0.6978
标志	0.1242	0.0521	0.0788	**0.2006**	0.1072
围栏	0.3028	0.3081	0.2760	**0.3726**	0.2970
车辆	0.4735	0.4612	0.4352	0.3728	**0.5025**
行人	0.1594	0.1095	0.1439	**0.1978**	0.1133
骑行者	0.4506	0.4701	0.4115	**0.5002**	0.4291

11 类里 DeepLabV3+ 拿到 9 类的最高 IoU。天空和车辆的最高值属于 Attention U-Net，分别是 0.9415 和 0.5025。

类别	模型	IoU	Precision	Recall
建筑	残差 U-Net	0.4433	0.8454	0.4825
建筑	DeepLabV3+	0.5971	0.9722	0.6075
建筑	SegNet	0.3342	0.8348	0.3579
车辆	Attention U-Net	0.5025	0.7214	0.6235
车辆	残差 U-Net	0.4735	0.6428	0.6426
车辆	DeepLabV3+	0.3728	0.4716	0.6401
行人	DeepLabV3+	0.1978	0.2051	0.8472
行人	残差 U-Net	0.1594	0.1674	0.7698
杆柱	DeepLabV3+	0.0500	0.0593	0.2412
杆柱	SegNet	0.0310	0.0366	0.1676
杆柱	无跳跃连接	0.0093	0.0161	0.0214

建筑是训练像素里第二多的类，也是五种模型拉开差距的主要位置。DeepLabV3+ 的建筑精确率 0.9722、召回率 0.6075：预测成建筑的区域很干净，召回也高于残差 U-Net 的 0.4825。SegNet 和无跳跃连接的建筑召回只有 0.36 左右，大块建筑被标成了别的类。

车辆上 DeepLabV3+ 的召回率 0.6401，与残差 U-Net 的 0.6426 接近，精确率只有 0.4716。它把不少不是车的像素也标成了车，所以车辆 IoU 反而最低。Attention U-Net 的车辆精确率 0.7214，是五种里最干净的。

行人都是高召回、低精确率。DeepLabV3+ 找回了 84.72% 的行人像素，但预测成行人的像素里只有 20.51% 是对的。类别权重把稀有类的损失放大之后，模型倾向于多标，IoU 仍然只有 0.1978。

杆柱在五种模型里都低于 0.05。无跳跃连接的召回率只有 0.0214。SegNet 靠池化索引把召回提到 0.1676，DeepLabV3+ 提到 0.2412，精确率仍然只有 0.0593。四次下采样后特征图只剩 15×20，一根杆往往不到一个像素宽。

8. 测试集可视化

下图是测试集中隔开选取的四帧。测试集是同一段视频，相邻帧几乎一样，所以按序号 0、36、60、84 取样，对应文件 0016E5_07959、0016E5_08031、0016E5_08079、0016E5_08127。四帧分别含前方车辆与骑行者、路口车辆、近处骑行者与行人、弯道上的近处骑行者。

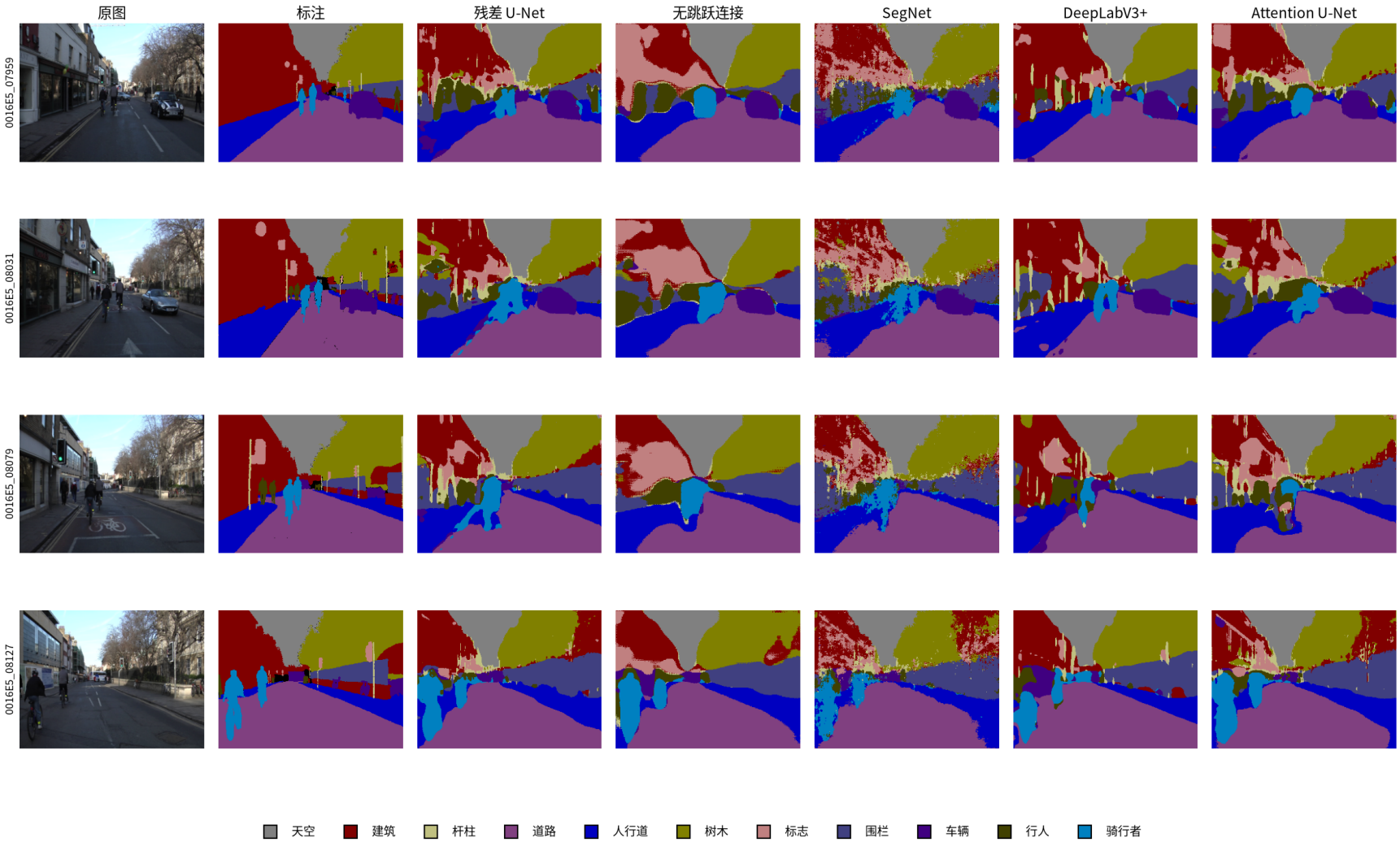


图 1　测试集四帧的分割结果。从左到右：原图、标注、残差 U-Net、无跳跃连接、SegNet、DeepLabV3+、Attention U-Net。灰色为天空，暗红为建筑，紫色为道路，蓝色为人行道，黄绿为树木，青色为骑行者，深紫色为车辆。

四帧里道路和天空在五种预测中都连成片，和总指标里这两类 IoU 高于 0.86 一致。建筑的差别最明显：DeepLabV3+ 的红色区域更整，另外四种在楼面和树冠交界处更容易碎成标志和树木的颜色。骑行者在四帧里都被标成青色块，轮廓比标注粗。杆柱在标注里是细竖线，五种预测都只剩下断续的短线。路口那一帧的白色车辆，Attention U-Net 和残差 U-Net 留下的车辆色块更集中，DeepLabV3+ 的车辆区域更容易散开，这和车辆 IoU 的排序一致。

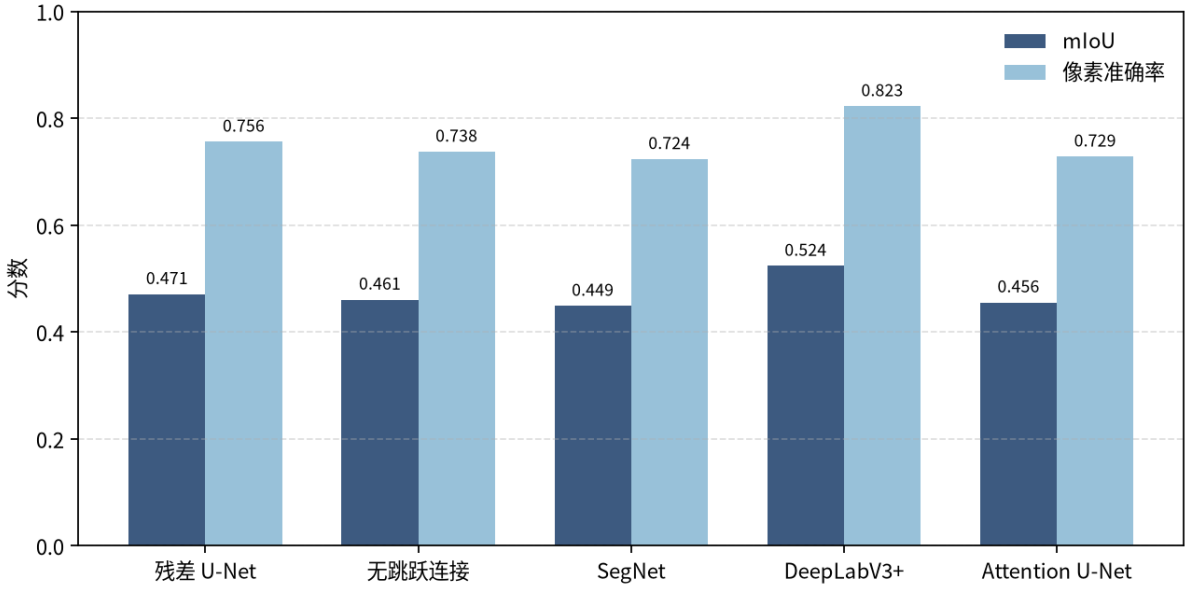


图 2　五种模型在测试集上的 mIoU 与像素准确率。

9. 结果分析

大面积类别已经学到。DeepLabV3+ 的道路 IoU 为 0.9181，树木为 0.8552，人行道为 0.7699，天空为 0.9286。像素准确率 0.8227 高于只预测道路所能达到的水平：道路约占训练像素的 29.84%。mIoU 为 0.5239，低于 PA，因为 11 个类等权平均，杆柱、标志和行人把均值拉低。FWIoU 为 0.7548，更接近 PA，说明按面积看，大类的重叠是好的。

DeepLabV3+ 的优势主要来自多尺度上下文。ASPP 在 15×20 的瓶颈上同时看局部膨胀卷积和整图平均，建筑、道路、树木、人行道、围栏、标志这些成片区域的标签更一致。它只从 1/4 分辨率接了一次低层特征，边界仍然粗，但已经够把建筑 IoU 从残差 U-Net 的 0.4433 提到 0.5971。这一头里还有 BatchNorm。归一化会让训练更稳，验证损失 0.5711 也低于残差 U-Net 的 0.8316。因此 0.0529 的 mIoU 差距里同时有空洞金字塔和归一化的作用。

跳跃连接的作用可以从残差 U-Net 和无跳跃连接的差看出来。mIoU 只差 0.0102（0.4710 对 0.4608），PA 差 0.0187。差距集中在建筑、杆柱、标志和行人。无跳跃连接的树木 IoU 为 0.8426，高于残差 U-Net 的 0.7244，接近 DeepLabV3+。树木是成片纹理，少了高分辨率边缘之后，标签更不容易被楼面和标志切碎。

SegNet 的 mIoU 最低，为 0.4491。它的道路 IoU 为 0.8786，高于残差 U-Net，杆柱也高于无跳跃连接。索引反池化能把强响应放回池化前的位置，所以细结构和路面边界还有保留。建筑 IoU 只有 0.3342，因为反池化不带回编码器的特征值，楼面这种需要高分辨率外观的区域补不回来。参数最少并没有换成更高的分类准确率。

Attention U-Net 的测试 mIoU 为 0.4555，低于残差 U-Net 0.0155；验证 mIoU 为 0.4449，也低于残差 U-Net 的 0.4500。注意力门压低了一部分跳跃特征，天空和车辆更干净，这两类 IoU 都是五种里最高的。建筑召回率从 0.4825 降到 0.3960，树木 IoU 从 0.7244 降到 0.6978。在只有 510 张训练图、又没有预训练的情况下，门控把有用的边缘一起压掉了，整体分类能力下降。

五种模型共同的失败是杆柱、标志和行人。标志 IoU 最高也只有 0.2006，行人最高 0.1978，杆柱最高 0.0500。这三类像素少、目标薄。加权交叉熵把召回拉高以后，精确率仍然很低，IoU 被误检拖住。要再提高这三类，需要更高的输入分辨率，或者专门的细目标结构。本次比较固定在 320×240。

10. 小结

实验在 CamVid 的 11 类语义分割上，用同一训练设置比较了残差 U-Net、无跳跃连接的编码器-解码器、SegNet、DeepLabV3+ 和 Attention U-Net。测试集上 DeepLabV3+ 的像素准确率为 0.8227，mIoU 为 0.5239，PA、mPA、mIoU、FWIoU、mDice 五项都是最高。11 类里它拿到 9 类的最高 IoU，包括建筑、道路、树木、人行道、标志、围栏、杆柱、行人和骑行者。残差 U-Net 排第二，mIoU 为 0.4710。去掉跳跃连接或改成池化索引上采样之后，mIoU 降到 0.4608 和 0.4491。注意力门控没有带来整体提升，mIoU 为 0.4555，只在天空和车辆上更好。道路和天空已经分得很稳，杆柱和行人的 IoU 仍然低。